

Aufgabe 5: Das ägyptische Grabmal

Team-ID: 00233

Team: Bougainvillea

Bearbeiter/-innen dieser Aufgabe:

Tao Zheng

14. Oktober 2024

Inhaltsverzeichnis

1 Lösungsidee	1
1.1 Zurücklaufen (Erweiterung)	2
2 Umsetzung	2
3 Beispiele	5
4 Quellcode (Python 3.11.9).	8

1 Lösungsidee

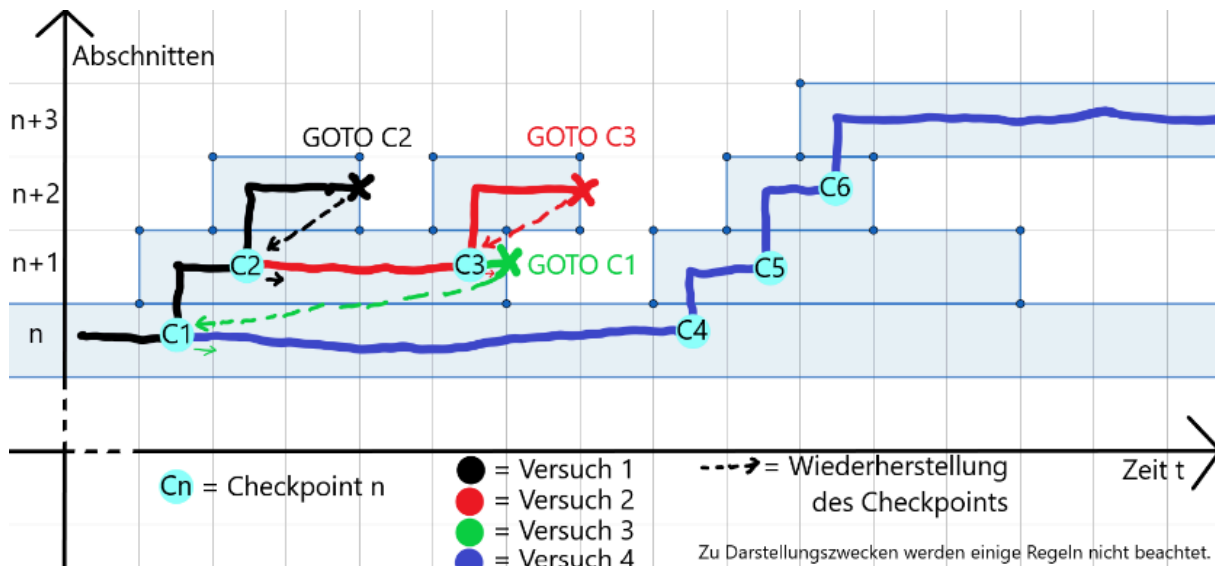
Meine Lösungsidee besteht darin, mithilfe der Brute-Force-Methode die zeitlich nächstliegenden Möglichkeiten auszuprobieren und dadurch die schnellste Lösung zu finden. Dabei wird versucht die Abschnitte zu betreten und wenn der aktuelle Abschnitt kurz vor dem Schließen steht, wird in den vorherigen Abschnitt zurückgegangen bzw. die zuletzt ausgeführten Schritte rückgängig gemacht, um die nächsten Möglichkeiten zu versuchen.

Jedes Mal, bevor man in den nächsten Abschnitt geht, wird ein Speicherpunkt erstellt. Bei den Speicherpunkten sollte dabei folgende Regel beachtet werden:

- Nach dem Wiederherstellen darf der nächste Abschnitt nicht sofort betreten werden, sondern man muss warten, bis sich der nächste Abschnitt einmal geschlossen und wieder geöffnet hat.
- Die Spielstände sollen ähnlich wie in einem Stapelspeicher gespeichert werden:
 - Jeder Speicherpunkt darf maximal nur einmal wiederhergestellt werden
 - Der zuletzt erstellte Speicherpunkt muss zuerst geladen werden, bevor ältere Speicherpunkte geladen werden (nach dem FILO-Prinzip).

Dabei werden die Zeitpunkte, die Positionen und die zuletzt ausgeführten Schritte gespeichert. Gerät man in eine Situation, wo sich der aktuelle Abschnitt schließen wird und man keine Möglichkeit hat rechtzeitig weiterzukommen oder es bereits erfolglos versucht wurde, lädt man den letzten Speicherpunkt. Dadurch übernimmt man die Zeit, Position und alle vorherigen Aktionen von dem Moment, bevor man den „gescheiterten“ Abschnitt betreten hat. Diese Schritte werden so lange wiederholt bis man das Ziel erreicht.

Das Beispiel im Bild unten zeigt die Vorgehensweise. Die y-Achse steht für die Abschnitte n , und die x-Achse für die Zeit t in Minuten. Zu Darstellungs Zwecken wurden einige Regeln außer Kraft gesetzt.



1.1 Zurücklaufen (Erweiterung)

Für das Zurücklaufen wird der Zeitpunkt des letzten Öffnungsmoments des vorherigen Abschnitts verwendet. Gerät man in eine Situation, wo sich der aktuelle Abschnitt schließt, geht man in der Zeit zurück und kehrt zum Öffnungszeitpunkt des vorherigen Abschnittes zurück. Gab es keinen Öffnungsmoment oder kommt man durch das Zurücklaufen nicht weiter, so wird der letzte noch nicht geladene Speicherpunkt geladen.

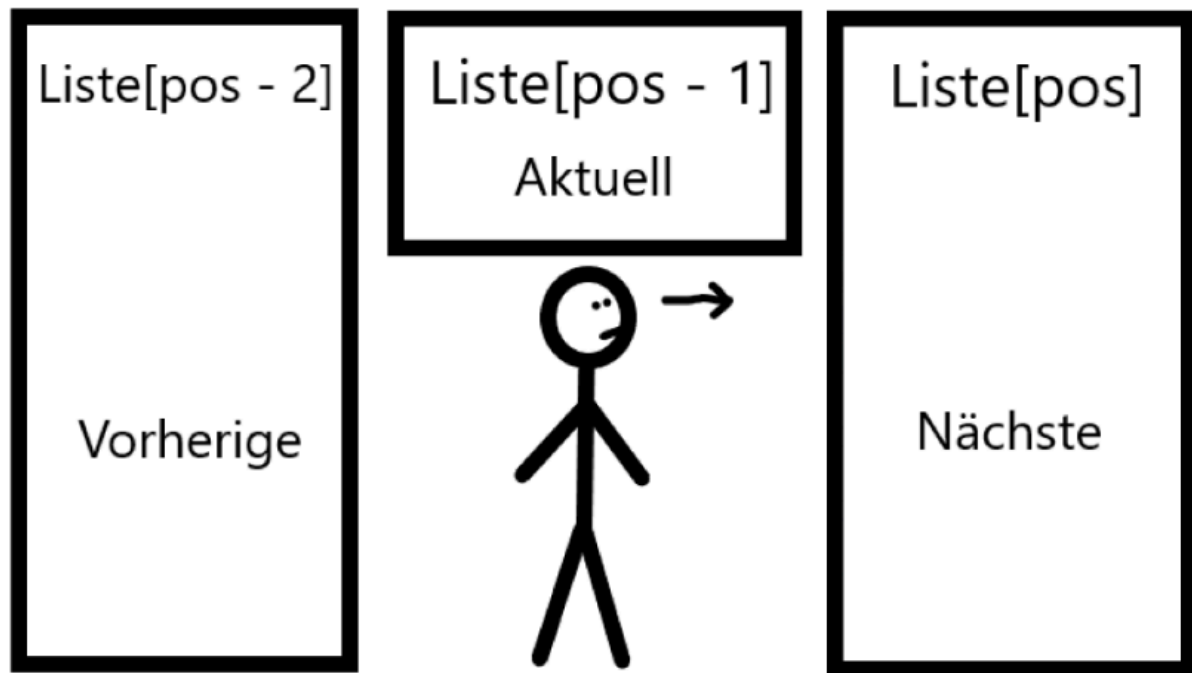
2 Umsetzung

Für die Umsetzung wird Python 3 verwendet. Die Abschnitte werden zunächst als eine eindimensionale Liste gespeichert, wobei jeder Abschnitt in einen Integer konvertiert wird. Anschließend wird die rekursive Funktion namens „Gehen“ erstellt, die einen Abschnitt darstellt. Die rekursive Funktion eignet sich meiner Meinung nach am besten, da alle Variablen für einen Abschnitt gespeichert werden können und bei der Rückkehr zu diesem Abschnitt wiederhergestellt werden. Die Funktion nimmt als erforderliche Argumente die Liste der Abschnitte (abschnitten) und die Anzahl der Abschnitte (gesamt_abschnitte) auf.

Da die Funktion rekursiv verwendet wird, gibt es optionale Parameter wie die aktuelle Position des Charakters (position), den Ankunftszeitpunkt (ankunft_zeit) und der Kontext, ob man in den vorherigen Abschnitt zurückgegangen ist (zurueck). Die position ist wichtig, um zu bestimmen in welchem Abschnitt man sich befindet, damit die korrekten Werte für die benachbarten Abschnitte abgelesen werden können. Der ankunft_zeit wird benötigt, um zu ermitteln wie viel Zeit bereits vergangen ist und um alle Abschnitte zu synchronisieren. Außerdem wird dieser Wert für die Berechnung von Wartezeiten verwendet. Der Kontext, ob man vorher in den vorherigen Abschnitt zurückgegangen ist, vermeidet dass man ohne warten sofort wieder nach vorne geht.

Für die Positionen gilt (vgl. Bild):

- Index Position minus 2 entspricht dem vorherigen Abschnitt,
- Index Position minus 1 entspricht dem aktuellen Abschnitt und
- Index Position unverändert entspricht dem nächsten Abschnitt.



`pos = position; Liste = abschnitten`

Zuerst prüft die Funktion, ob das Ziel erreicht wurde, indem man die aktuelle Position mit der Anzahl der Abschnitte vergleicht und schaut, ob es sich übereinstimmt. Ebenso überprüft es, ob der nächste Abschnitt bereits offen ist, um diesen direkt zu überspringen (außer man ist zuvor zurückgegangen). Dies lässt sich überprüfen, indem der ganzzahlige Quotient aus der Ankunftszeit und der Dauer des Abschnitts gebildet und geprüft wird, ob dieser nicht durch zwei teilbar ist. Ist dies nicht der Fall, muss man im aktuellen Abschnitt warten. Die Ankunftszeit wird in einer neuen Variablen (`aktuelle_zeit`) gespeichert, um später die Wartezeit durch die Differenz der beiden Werte berechnen zu können.

Mithilfe der Position überprüft die Funktion anschließend, ob der aktuelle (`position > 0`) und der vorherige Abschnitt (`position > 1`) sich bewegen könnten. Ist dies der Fall, werden die Dauern der Abschnitte (`aktuelle_abschnitt_dauer`, `vorherige_abschnitt_dauer`) ausgelesen und berücksichtigt. Diese werden später benötigt, um herauszufinden, wann der aktuelle Abschnitt schließt und wann die letzte Möglichkeit besteht, in den vorherigen Abschnitt zurückzugehen.

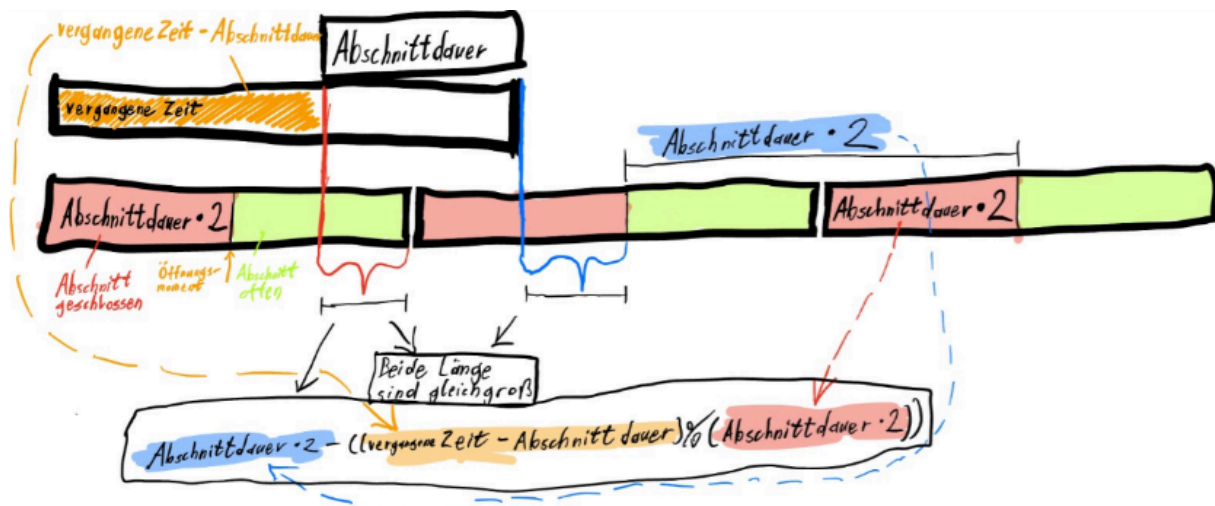
Die Funktion tritt nun in eine Schleife ein, die nur bei Rückgabe eines Wertes gebrochen wird. Mithilfe der folgenden Formel:

$$\text{Dauer bis zur nächsten Abschnittsaktion} = \text{Abschnittsdauer} - (\text{vergangene Zeit} \% \text{Abschnittsdauer})1$$



lässt sich berechnen wie viele Minuten man warten muss, bis sich ein bestimmter Abschnitt öffnet oder schließt. Die „Abschnittsdauer“, steht für die Zeitspanne, die man abwarten muss, bis sich der Abschnitt öffnet oder schließt, und die „vergangene Zeit“ ist die bisher verstrichene Zeit. Diese Berechnung lässt sich auch auf Abschnittöffnungen erweitern:

$$\begin{aligned} & \text{Dauer bis zur nächsten Abschnittöffnungszeitpunkt} = \\ & = \text{Abschnittsdauer} \cdot 2 - ((\text{vergangene Zeit} - \text{Abschnittsdauer}) \% (\text{Abschnittsdauer} \cdot 2)) \end{aligned} \quad (2)$$



Die Formel für die Abschnittsöffnung wird sowohl für Abschnitte verwendet, die noch betreten werden müssen, als auch für den vorherigen Abschnitt (z. B. `zeit_bis_vorherige_abschnitt` und `zeit_bis_naechster_abschnitt`). Die Formel für die Abschnittsaktion wird für den aktuellen Abschnitt verwendet (`verbleibende_zeit_aktueller_abschnitt`), da dieser bereits geöffnet ist und daher klar ist, dass die nächste Aktion die Schließung des Abschnitts sein wird.

Zunächst wird überprüft, ob man zurückgehen kann. Dafür benötigt man die Dauer bis zur nächsten Abschnittsöffnung des vorherigen Abschnitts und prüft, ob der Öffnungsmoment vor der Schließung des aktuellen Abschnitts liegt und ob man bereits zuvor zurückgegangen ist oder der vorherige Abschnitt schon einmal geschlossen wurde (`nicht_gleiche_abschnitt`). Um zu überprüfen, ob der Abschnitt bereits einmal geschlossen war, muss der Öffnungsmoment berechnet werden (`zeit_bis_vorherige_abschnitt + aktuelle_zeit`) und dann von der Ankunftszeit (`ankunft_zeit`) subtrahiert werden. Anschließend wird geprüft, ob diese Differenz größer ist als die Dauer des vorherigen Abschnitts. Wenn beide Bedingungen erfüllt sind, wird dieser Zeitpunkt als `vorherige_abschnitt_moment` gespeichert.

(Kommentar: Ich habe mich bewusst dagegen entschieden, das letzte Öffnungsmoment des vorherigen Abschnitts kurz vor der Abschnittschließung von dem aktuellen Abschnitt berechnen zu lassen. Als ich

diese Lösung ausprobiert hatte, wurde die Ausführungsdauer für Grabmal 5 von 0.79s auf 1.XXs erhöht. Außerdem müsste ich einige Ausnahmefälle einbauen, um Probleme zu vermeiden, was den Code noch komplexer gemacht hätte. Aus diesem Grund habe ich diese Idee verworfen und die einfachere Variante implementiert.)

Nun werden die Zeiten für die Öffnung des nächsten Abschnitts und die Schließung des aktuellen Abschnitts miteinander verglichen, ob es noch möglich ist den nächsten Abschnitt zu betreten, bevor der aktuelle Abschnitt schließt (falls der aktuelle Abschnitt sich bewegen kann). Ist die Dauer bis zur nächsten Abschnittsaktion des aktuellen Abschnitts größer als die des nächsten Abschnitts, wird die verstrichene Zeit um die Öffnungsdauer des nächsten Abschnitts (also der Differenz zwischen der vergangenen Zeit und dem nächsten Öffnungsmoment) erhöht. Anschließend wird die Funktion `Gehen` erneut aufgerufen, wobei alle erforderlichen Argumente unverändert bleiben, außer dass die Position um eins erhöht und die aktuelle Zeit weitergegeben wird.

Ist die Abschnittsaktionsdauer des aktuellen Abschnitts kleiner oder gleich der Öffnungsdauer des nächsten Abschnitts, wird der Versuch abgebrochen und die Funktion gibt eine leere Menge zurück oder man geht in den vorherigen Abschnitt zurück, falls ein Öffnungszeitpunkt für den vorherigen Abschnitt existiert. Jedes Mal wird geprüft, ob die Funktion `Gehen` eine leere Menge zurückgibt. Gibt die aufgerufene Funktion eine leere Menge zurück, signalisiert dies der vorherigen Funktion, dass dieser Weg nicht funktioniert hat und eine andere Option ausprobieren soll, indem sich die Schleife von vorn beginnt.

Wird ein funktionierender Weg gefunden, gibt die zuletzt aufgerufene Funktion einen Rückgabewert in Form eines Strings mit Text, welcher die Position und Ankunftszeit beinhaltet, an die anderen Funktionen weiter. Die restlichen Funktionen fügen dann die Dauer und die entsprechende Position hinzu und geben diese Informationen weiter. Ist man weitergekommen, ohne dass Zeit im Abschnitt vergangen ist, wird der Text unverändert weitergegeben, da ein anderer Abschnitt es schon bereits übernimmt. Wurde ein Abschnitt betreten, fügt man am Ende die Dauer und den aktuellen Abschnitt hinzu. Wurde zurückgegangen, wird das Gleiche hinzugefügt jedoch mit der Information, dass man zurückgehen musste.

Am Ende erhält die zuerst aufgerufene Funktion eine vollständige Anweisung, wie man sich durch die Abschnitte bewegen muss.

3 Beispiele

Die Ausführdauer der Funktion wird wie folgt ausgerechnet:

- `start = time.time(); Gehen(...); end = time.time() - start`

Das bedeutet, dass für die Dauer nur die tatsächliche Ausführung der Funktion zählt und nicht der Funktionsdefinitionsbereich.

Für das Grabmal 0 wurde eine Gesamtdauer von 12 Minuten ausgerechnet (Ausführdauer: 0.0000s), mit der Anweisung:

- Warte 5 Minuten. Geh zu Abschnitt: 1.
- Warte 3 Minuten. Geh zu Abschnitt: 2.
- Warte 4 Minuten. Geh zu Abschnitt: 3.
- Du bist angekommen. (Gesamtdauer: 12 Minuten)

Dabei wurde die Funktion `Gehen` insgesamt 4-mal aufgerufen, mit folgenden optionalen Argumenten:

- `Gehen(position=0, ankunft_zeit=0)`
- `Gehen(position=1, ankunft_zeit=5)`
- `Gehen(position=2, ankunft_zeit=8)`
- `Gehen(position=3, ankunft_zeit=12)`

Das bedeutet, dass in diesem Fall ein direkter Weg gefunden wurde, ohne dass es zu einer aussichtslosen Situation kam.

Für das Grabmal 1 wurde eine Gesamtdauer von 49 Minuten ausgerechnet (Ausführdauer: 0.0000s), mit der Anweisung:

- Warte 17 Minuten. Geh zu Abschnitt: 2.
- Warte 4 Minuten. Geh zu Abschnitt: 3.
- Warte 6 Minuten. Geh zu Abschnitt: 4.
- Warte 8 Minuten. Geh zurueck zu Abschnitt: 3.
- Warte 4 Minuten. Geh zurueck zu Abschnitt: 2.
- Warte 10 Minuten. Geh zu Abschnitt: 5.
- Du bist angekommen. (Gesamtdauer: 49 Minuten)

In diesem Fall wurde die Funktion Gehen 10 mal aufgerufen.:

- Gehen(position=0, ankunft_zeit=0)
- Gehen(position=1, ankunft_zeit=17)
- Gehen(position=2, ankunft_zeit=17)
- Gehen(position=3, ankunft_zeit=21)
- Gehen(position=4, ankunft_zeit=27)
- Gehen(position=3, ankunft_zeit=35, zurueck=True)
- Gehen(position=2, ankunft_zeit=39, zurueck=True)
- Gehen(position=3, ankunft_zeit=49)
- Gehen(position=4, ankunft_zeit=49)
- Gehen(position=5, ankunft_zeit=49)

Daraus wird deutlich, dass man im Abschnitt 1 (position = 1) direkt weitergehen konnte, weshalb Abschnitt 1 nicht ausgegeben wurde und man bei Minute 17 direkt zu Abschnitt 2 übergeht. Außerdem gab es im Abschnitt 4 die Möglichkeit, zurückzugehen, was letztlich durchgeführt wurde und zum Ziel führte.

Für das Grabmal 2 wurde eine Gesamtdauer von 490021 Minuten ausgerechnet (Ausführdauer: 0.000s), mit der Anweisung:

- Warte 170000 Minuten. Geh zu Abschnitt: 2.
- Warte 40009 Minuten. Geh zu Abschnitt: 3.
- Warte 59991 Minuten. Geh zu Abschnitt: 4.
- Warte 80015 Minuten. Geh zurueck zu Abschnitt: 3.
- Warte 39985 Minuten. Geh zurueck zu Abschnitt: 2.
- Warte 100021 Minuten. Geh zu Abschnitt: 5.
- Du bist angekommen. (Gesamtdauer: 490021 Minuten)

Da die Ausführungsdauer 0.0s beträgt, lässt sich daraus schließen, dass diese Implementierung der Lösung auch bei großen Zahlen effizient funktioniert.

Für das Grabmal 3 wurde eine Gesamtdauer von 35 Minuten ausgerechnet (Ausführdauer: 0.0000s).

- Warte 20 Minuten. Geh zu Abschnitt: 1.
- Warte 2 Minuten. Geh zu Abschnitt: 6.
- Warte 13 Minuten. Geh zu Abschnitt: 10.

Du bist angekommen. (Gesamtdauer: 35 Minuten)

Für Grabmal 3 wurde Funktion Gehen 24 mal ausgelöst.:

- Gehen(position=0, ankunft_zeit=0)
- Gehen(position=1, ankunft_zeit=4)
- Gehen(position=1, ankunft_zeit=12)
- Gehen(position=1, ankunft_zeit=20)
- Gehen(position=2, ankunft_zeit=22)
- Gehen(position=3, ankunft_zeit=22)
- Gehen(position=4, ankunft_zeit=22)

- `Gehen(position=5, ankunft_zeit=22)`
- `Gehen(position=6, ankunft_zeit=22)`
- `Gehen(position=7, ankunft_zeit=23)`
- `Gehen(position=8, ankunft_zeit=23)`
- `Gehen(position=9, ankunft_zeit=23)`
- `Gehen(position=7, ankunft_zeit=25, zurueck=True)`
- `Gehen(position=7, ankunft_zeit=25)`
- `Gehen(position=8, ankunft_zeit=25)`
- `Gehen(position=7, ankunft_zeit=27, zurueck=True)`
- `Gehen(position=7, ankunft_zeit=27)`
- `Gehen(position=8, ankunft_zeit=27)`
- `Gehen(position=7, ankunft_zeit=29)`
- `Gehen(position=7, ankunft_zeit=31)`
- `Gehen(position=7, ankunft_zeit=33)`
- `Gehen(position=7, ankunft_zeit=35)`
- `Gehen(position=8, ankunft_zeit=35)`
- `Gehen(position=9, ankunft_zeit=35)`
- `Gehen(position=10, ankunft_zeit=35)`

Hier wird deutlich, wie mit der Brute-Force-Methode nach einer Möglichkeit gesucht wird. Bis zur 23te Minute passt alles. Anschließend gibt Abschnitt 9 eine leere Menge zurück, und Abschnitt 8 versucht, über den Rückweg eine Lösung zu finden. Dies schlägt jedoch fehl, und der Versuch wird wiederholt, die 7te Abschnitt zu betreten, bis es schließlich bei Minute 35 klappt und der Weg, bis Abschnitt 10 durchlaufen werden kann.

Für das Grabmal 4 wurde eine Gesamtdauer von 3432897 Minuten ausgerechnet (Ausführdauer: 0.002s).

- Warte 3242835 Minuten. Geh zu Abschnitt: 1.
- Warte 60360 Minuten. Geh zu Abschnitt: 2.
- Warte 360 Minuten. Geh zu Abschnitt: 5.
- Warte 37695 Minuten. Geh zu Abschnitt: 6.
- Warte 51407 Minuten. Geh zu Abschnitt: 8.
- Warte 36216 Minuten. Geh zu Abschnitt: 9.
- Warte 4024 Minuten. Geh zu Abschnitt: 10.
- Du bist angekommen. (Gesamtdauer: 3432897 Minuten)

Für das Grabmal 5 wurde eine Gesamtdauer von 126180177 Minuten ausgerechnet (Ausführdauer: 0.781s).

- Warte 126017460 Minuten. Geh zu Abschnitt: 1.
- Warte 5507 Minuten. Geh zu Abschnitt: 3.
- Warte 196 Minuten. Geh zu Abschnitt: 4.
- Warte 4 Minuten. Geh zu Abschnitt: 5.
- Warte 7864 Minuten. Geh zu Abschnitt: 6.
- Warte 3138 Minuten. Geh zu Abschnitt: 7.
- Warte 4348 Minuten. Geh zu Abschnitt: 8.
- Warte 3960 Minuten. Geh zu Abschnitt: 9.
- Warte 3087 Minuten. Geh zu Abschnitt: 10.
- Warte 3996 Minuten. Geh zu Abschnitt: 13.
- Warte 2169 Minuten. Geh zu Abschnitt: 14.
- Warte 3152 Minuten. Geh zu Abschnitt: 15.
- Warte 968 Minuten. Geh zu Abschnitt: 16.
- Warte 1264 Minuten. Geh zu Abschnitt: 17.
- Warte 6120 Minuten. Geh zu Abschnitt: 18.
- Warte 3799 Minuten. Geh zu Abschnitt: 19.

- Warte 4095 Minuten. Geh zurueck zu Abschnitt: 18.
- Warte 1008 Minuten. Geh zurueck zu Abschnitt: 17.
- Warte 6886 Minuten. Geh zu Abschnitt: 18.
- Warte 1115 Minuten. Geh zu Abschnitt: 19.
- Warte 784 Minuten. Geh zu Abschnitt: 21.
- Warte 2840 Minuten. Geh zu Abschnitt: 23.
- Warte 2109 Minuten. Geh zu Abschnitt: 25.
- Warte 641 Minuten. Geh zu Abschnitt: 26.
- Warte 2614 Minuten. Geh zu Abschnitt: 28.
- Warte 21 Minuten. Geh zu Abschnitt: 30.
- Warte 4872 Minuten. Geh zu Abschnitt: 31.
- Warte 7755 Minuten. Geh zu Abschnitt: 32.
- Warte 403 Minuten. Geh zu Abschnitt: 36.
- Warte 3623 Minuten. Geh zu Abschnitt: 38.
- Warte 2913 Minuten. Geh zu Abschnitt: 39.
- Warte 5449 Minuten. Geh zu Abschnitt: 42.
- Warte 2698 Minuten. Geh zu Abschnitt: 43.
- Warte 3850 Minuten. Geh zu Abschnitt: 44.
- Warte 1087 Minuten. Geh zu Abschnitt: 50.
- Warte 2086 Minuten. Geh zu Abschnitt: 53.
- Warte 3763 Minuten. Geh zurueck zu Abschnitt: 52.
- Warte 3091 Minuten. Geh zurueck zu Abschnitt: 51.
- Warte 3365 Minuten. Geh zu Abschnitt: 52.
- Warte 2435 Minuten. Geh zu Abschnitt: 53.
- Warte 5008 Minuten. Geh zu Abschnitt: 54.
- Warte 1428 Minuten. Geh zu Abschnitt: 55.
- Warte 8625 Minuten. Geh zu Abschnitt: 56.
- Warte 3114 Minuten. Geh zu Abschnitt: 57.
- Warte 103 Minuten. Geh zu Abschnitt: 63.
- Warte 1097 Minuten. Geh zu Abschnitt: 66.
- Warte 103 Minuten. Geh zu Abschnitt: 68.
- Warte 1462 Minuten. Geh zu Abschnitt: 69.
- Warte 3643 Minuten. Geh zu Abschnitt: 71.
- Warte 4843 Minuten. Geh zu Abschnitt: 72.
- Warte 1829 Minuten. Geh zu Abschnitt: 73.
- Warte 567 Minuten. Geh zu Abschnitt: 75.
- Warte 5418 Minuten. Geh zu Abschnitt: 77.
- Warte 4662 Minuten. Geh zu Abschnitt: 83.
- Warte 1740 Minuten. Geh zu Abschnitt: 85.
- Du bist angekommen. (Gesamtdauer: 126180177 Minuten)

Somit ist dieses Programm auch für lange Listen mit großen Zahlenwerten effektiv. Im Programm selbst habe ich keine weiteren Optimierungsmöglichkeiten (ohne dass die Lesbarkeit des Quellcodes verringert wird) gefunden. Man sollte außerdem noch bedenken, dass die Ausführdauer von der Leistung des Gerätes abhängig ist. Die Ausführdatei ist aktuell auf die grabmal5.txt Datei eingestellt.

Es ist außerdem wichtig zu erwähnen, dass Python für mehr als 1000 Abschnitten die RecursionError auslöst da Python ein eingebautes Recursion-Limit hat. Es ist wichtig bei langen Listen das Recursion Limit über die Funktion `sys.setrecursionlimit` zu erhöhen.

4 Quellcode (Python 3.11.9).

```
def Gehen(abschnitten, gesamt_abschnitte, position=0, ankunft_zeit=0, zurueck=False) -> str:
```

```

    #print(f"Gehen(position={position}, ankunft_zeit={ankunft_zeit}{f', zurueck={zurueck}' if
zurueck else ''})")
    """
    Berechnet den Weg durch eine Serie von Abschnitten mit unterschiedlicher Dauer.

    Argumente:
    - abschnitten (list): Liste der Dauer der Abschnitte.
    - gesamt_abschnitte (int): Gesamtanzahl der Abschnitte.
    - position (int): Der aktuelle Abschnitt
    - ankunft_zeit (int): Die Zeit beim Betreten des Abschnittes
    - zurueck (bool): Kontext, ob man eine sofortige nach vorne laufen vermeiden sollte.

    Rückgabewerte:
    - str: Beschreibung der Schritte zum Ziel oder leere Menge bei falschem Wege.
    """

    # Basisfall: Wenn alle Abschnitte durchlaufen wurden.
    if position == gesamt_abschnitte:
        return f"{position}.\nDu bist angekommen. (Gesamtdauer: {ankunft_zeit} Minuten)"

    # Prüfen ob der aktuelle Abschnitt offen ist.
    if not zurueck and (ankunft_zeit // abschnitten[position]) % 2:
        schritte = Gehen(abschnitten, gesamt_abschnitte, position + 1, ankunft_zeit )
        if schritte != "":
            return schritte

    # Starte bei der aktuellen Zeit.
    aktuelle_zeit = ankunft_zeit

    naechste_abschnitt_dauer = abschnitten[position]

    # Überprüfen, ob man sich in einem Abschnitt befindet.
    aktuelle_abschnitt_dauer = 0
    if position != 0:
        aktuelle_abschnitt_dauer = abschnitten[position - 1]

    # Überprüfen, ob es einen vorherigen Abschnitt existiert.
    vorherige_abschnitt_moment = None
    vorherige_abschnitt_dauer = 0
    if position > 1:
        vorherige_abschnitt_dauer = abschnitten[position - 2]

    while True:
        # Berechne die Zeit bis der nächste Abschnitt wieder geöffnet wird
        zeit_bis_naechster_abschnitt = naechste_abschnitt_dauer*2 - ((aktuelle_zeit -
naechste_abschnitt_dauer) % (naechste_abschnitt_dauer*2))

        # Prüfen, ob der aktuelle Abschnitt schließt, bevor der nächste geöffnet wird
        if aktuelle_abschnitt_dauer > 0:
            verbleibende_zeit_aktueller_abschnitt = aktuelle_abschnitt_dauer - (aktuelle_zeit
% aktuelle_abschnitt_dauer)

            if vorherige_abschnitt_dauer > 0:
                zeit_bis_vorherige_abschnitt = vorherige_abschnitt_dauer*2 - ((aktuelle_zeit -
vorherige_abschnitt_dauer) % (vorherige_abschnitt_dauer*2))
                nicht_gleiche_abschnitt = (zeit_bis_vorherige_abschnitt + aktuelle_zeit -
ankunft_zeit) > vorherige_abschnitt_dauer
                if verbleibende_zeit_aktueller_abschnitt > zeit_bis_vorherige_abschnitt and
(zurueck or nicht_gleiche_abschnitt):
                    vorherige_abschnitt_moment = zeit_bis_vorherige_abschnitt + aktuelle_zeit

            if verbleibende_zeit_aktueller_abschnitt <= zeit_bis_naechster_abschnitt:

```

```
        if not vorherige_abschnitt_moment is None:
            schritte = Gehen(abschnitten, gesamt_abschnitte, position - 1,
vorherige_abschnitt_moment, True)
            if schritte != "":
                wartezeit = vorherige_abschnitt_moment - ankunft_zeit
                return f"{position}.\nWarte {wartezeit} Minuten. Geh zurueck zu
Abschnitt: " + schritte
            return "" # Der Weg funktioniert zeitlich nicht. Geh zurück!

aktuelle_zeit += zeit_bis_naechster_abschnitt

# Geh zum nächsten Abschnitt
schritte = Gehen(abschnitten, gesamt_abschnitte, position + 1, aktuelle_zeit)
if schritte != "":
    wartezeit = aktuelle_zeit - ankunft_zeit
    if position == 0:
        return f"Warte {wartezeit} Minuten. Geh zu Abschnitt: " + schritte
    return f"{position}.\nWarte {wartezeit} Minuten. Geh zu Abschnitt: " + schritte
```